

```

class Solution(object):
    def isPalindrome(self, head):
        slow, fast = head, head

        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next

        curr = slow
        prev = None
        while curr != None:
            next_node = curr.next
            curr.next = prev
            prev = curr
            curr = next_node

        first = head
        second = prev
        while second != None:
            if first.val != second.val:
                return False
            first = first.next
            second = second.next

        return True

```

```

class ListNode(object):
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

    # Helper to convert a Python list to a linked list
    @staticmethod
    def from_list(arr):
        if not arr: return None
        head = ListNode(arr[0])
        current = head
        for val in arr[1:]:
            current.next = ListNode(val)
            current = current.next
        return head

    # Helper to convert a linked list to a Python list for printing
    def to_list(self):
        arr = []
        current = self
        while current:
            arr.append(current.val)
            current = current.next
        return arr

class Solution(object):
    def mergeTwoLists(self, list1, list2):

        dummy = ListNode(0)
        tail = dummy

        while list1 and list2:
            if list1.val < list2.val:
                tail.next = list1
                list1 = list1.next
            else:
                tail.next = list2
                list2 = list2.next
            tail = tail.next

        if list1:
            tail.next = list1
        elif list2:
            tail.next = list2

        return dummy.next

```

```

class Solution(object):
    def rotateRight(self, head, k):
        if not head or not head.next:
            return head

        curr, length = head, 1
        while curr.next:
            curr = curr.next
            length += 1

        k %= length
        if k == 0:
            return head

        curr.next = head

        for _ in range(length - k):
            curr = curr.next

        new_head = curr.next
        curr.next = None
        return new_head

```

```

class Solution(object):
    def addTwoNumbers(self, l1, l2):
        #l1, l2 = l1.next, l2.next
        carry = 0
        head = ListNode()
        curr = head
        while l1 or l2:
            val1 = l1.val if l1 else 0
            val2 = l2.val if l2 else 0
            sum = val1 + val2 + carry
            carry = sum // 10
            curr.next = ListNode(sum % 10)
            curr = curr.next
            l1 = l1.next if l1 else None
            l2 = l2.next if l2 else None
        if carry:
            curr.next = ListNode(carry)
        return head.next

```

```

import numpy as np

n = tuple(map(int, input().split()))
print(np.zeros(n, dtype=int))
print(np.ones(n, dtype=int))

```

```

3 3 3
[[[0 0 0]
  [0 0 0]
  [0 0 0]]

 [[0 0 0]
  [0 0 0]
  [0 0 0]]

 [[0 0 0]
  [0 0 0]
  [0 0 0]]]
[[[1 1 1]
  [1 1 1]
  [1 1 1]]

 [[1 1 1]
  [1 1 1]
  [1 1 1]]

 [[1 1 1]
  [1 1 1]
  [1 1 1]]]

```

```

class Solution(object):
    def addTwoNumbers(self, l1, l2):
        dummy = ListNode()
        curr = dummy
        carry = 0

```

```
carry = 0
while l1 or l2 or carry:
    val1 = l1.val if l1 else 0
    val2 = l2.val if l2 else 0
    sum = val1 + val2 + carry
    carry = sum // 10
    curr.next = ListNode(sum % 10)
    curr = curr.next
    l1 = l1.next if l1 else None
    l2 = l2.next if l2 else None
    if carry:
        curr.next = ListNode(carry)
return dummy.next
```